

Evaluación 3 ECSD 2026-1

Ingeniería Civil Mecatrónica USACH

Versión 1.0

Última Modificación: 27/07/2026

Objetivos

Al finalizar la actividad, el equipo deberá demostrar que puede:

1. **Diseñar un Sistema Distribuido (SD) simple**, identificando servicios, responsabilidades, interfaces y dependencias.
2. **Implementar comunicación entre servicios** (p. ej., HTTP/REST o mensajería simple) con serialización consistente (JSON) y validación básica.
3. **Construir una UI operativa** que permita **operar el sistema** (crear consultas/acciones, visualizar estados/resultados) y que consuma servicios remotos.
4. **Desplegar el sistema en un entorno reproducible** (virtualización, contenedores o hardware real), incluyendo instrucciones claras para ejecución.
5. **Aplicar buenas prácticas de Ingeniería de Software** a nivel básico: estructura del repositorio, configuración por entorno, logging, y documentación.
6. **Comunicar resultados** mediante **presentación, informe técnico, código fuente** y un **demo** funcional.

Reglas Generales

1. Los trabajos deben ser desarrollados en grupos de 3 a 4 estudiantes, utilizando el tiempo autónomo de trabajo considerado en el programa del curso. Los entregables de esta evaluación son un informe escrito, una presentación oral, una aplicación y una demostración de dicha aplicación.
2. El informe y la presentación deben ser legibles y ordenados, procurando utilizar los formatos disponibles en el Departamento de Ingeniería Mecánica.
3. Extensión máxima del informe escrito: 15 páginas de contenido (excluyendo portada, índice y referencias) + 15 páginas de anexos (figuras y tablas).
4. Extensión máxima de la presentación oral: 15 minutos. Todos los integrantes deben presentar.
5. Todas las citas a material disponible en la literatura (páginas web, papers, libros) deben estar correctamente referenciadas utilizando formato APA o IEEE. Cualquier material incorrectamente citado será considerado como plagio.
6. Está prohibido el uso de IA Generativa (ej: ChatGPT, Gemini, Claude) para la generación de texto para el informe y/o generación de código fuente (en caso de haberlo).
7. Se dejará en el curso de ECSD de Campus Virtual una copia digital de este documento, más la rúbrica de evaluación. El documento digital de enunciado puede recibir cambios durante el desarrollo de la evaluación, en dicho caso, la versión enlazada en Campus Virtual siempre será considerada como válida. Procure visitar el enunciado en caso de haber cambios.

(para el caso de las Reglas 5 y 6, cualquier infracción a ellas será evaluado con nota final mínima, sin derecho a apelación)

FECHA LÍMITE SUBIDA ENTREGABLES: Lunes 24/08/2026 a las 11:30 hrs.

(se debe entregar tanto informe como slides de presentación, ambos en formato PDF, más un archivo comprimido .zip con el código fuente de su aplicación)

Descripción de la Evaluación

Contexto general

El objetivo es **construir y desplegar un Sistema Distribuido simple** que resuelva uno de los enunciados propuestos, incorporando una **UI obligatoria**.

Requisitos mínimos del sistema

El sistema debe cumplir con:

- Arquitectura distribuida con al menos 3 servicios/procesos.
- Ejecución en al menos 2 nodos lógicos (por ejemplo, 2 contenedores; o 1 VM + host; o 2 máquinas; etc.).
- **UI obligatoria:**
 - Puede ser Streamlit (recomendado) u otra UI equivalente (web).
 - La UI debe consumir al menos un servicio remoto del SD y permitir interacción real (no solo visualización estática).
- **Persistencia obligatoria** (una de las siguientes):
 - Base de datos (MySQL/SQLite según el caso), o
 - Almacenamiento estructurado en archivos (JSON/CSV) solo si está justificado por el enunciado.
- **Documentación y reproducibilidad:**
 - Un comando o procedimiento estándar para levantar todo el sistema (idealmente `docker compose up`, o scripts claros si es VM/hardware).
 - README con instalación, configuración y ejecución.

Notas:

- No se requiere tolerancia a fallos (reintentos, circuit breaker, etc.). Sí se espera manejo básico de errores (validación de entradas, respuestas HTTP adecuadas, mensajes claros en UI).
- De ser necesario, puede hacer supuestos para resolver aspectos que no estén especificados en el enunciado, todos los supuestos que haga deben estar documentados en el informe y ser explicados durante el demo.

Tecnologías permitidas

Los equipos pueden elegir su stack, manteniendo interoperabilidad:

- **Python:** FastAPI/Flask, Streamlit, requests/httpx.
- **Java:** Spring Boot / Jakarta EE (según preferencia), clientes HTTP estándar.
- Comunicación: **REST/HTTP + JSON** (recomendado); mensajería opcional si el equipo lo domina.
- Despliegue: **Docker / Docker Compose** (recomendado), **VMs** (VirtualBox), o **hardware real**.
- Persistencia: MySQL / SQLite / archivos.

Cualquier otra tecnología que desee utilizar debe ser consultada con el profesor.

Caso 1 — Flota de Drones Industriales con Tolerancia a Fallos

Contexto

Coordinación de una flota de drones de inspección en una planta industrial, donde la pérdida de conectividad con un dron o con un servicio no puede detener el sistema completo.

Descripción del SD

Sistema que recibe telemetría de múltiples drones simulados, la enruta mediante un broker de mensajes, la procesa y permite asignar misiones desde una UI, tolerando la caída temporal de cualquier servicio.

Servicios mínimos

1. **Servicio de Drones**
 - Simula N drones publicando telemetría (posición, batería, estado).
2. **Servicio de Coordinación de Misiones**
 - Consume telemetría, asigna misiones, detecta drones sin respuesta.
3. **Servicio de Registro y Auditoría**
 - Persiste eventos e historial de misiones del sistema.
4. **Servicio de Notificaciones**
 - Alerta ante batería crítica o pérdida de contacto.

UI obligatoria

- Mapa/lista de drones en tiempo casi real.
- Panel de asignación de misiones.
- Visualización de qué servicios están saludables (health check).

Distribución

- Drones en un nodo.
- Broker + coordinación en otro nodo.
- Registro + Notificaciones en otro nodo.

Caso 2 — Línea de Producción con Balanceo Dinámico

Contexto

Línea de ensamblaje donde el cuello de botella puede desplazarse entre estaciones, y el sistema debe repartir carga entre réplicas de una misma estación.

Descripción del SD

Sistema que ejecuta al menos una estación replicada en 2+ instancias detrás de un balanceador, coordina el reparto de órdenes evitando condiciones de carrera, y detecta cuellos de botella en vivo.

Servicios mínimos

1. **Servicio de Órdenes**
 - Crea y encola órdenes de producción.
2. **Servicio de Estación**
 - Procesa órdenes con tiempos configurables.
 - Clasifica estado: normal / advertencia / crítico.
3. **Servicio de Configuración de Carga**
 - Usa un lock/contador distribuido (por ejemplo: Redis) para que dos réplicas no tomen la misma orden.
4. **Servicio de Métricas**
 - Calcula lead time y detecta cuello de botella por estación.

UI obligatoria

- Timeline de producción.
- Indicador de carga por réplica.
- Alerta visual de cuello de botella.

Distribución

- Órdenes + Balanceador.
- Estación de réplicas

- Coordinación de carga + Métricas.

Caso 3 — Gemelo Digital de Celda Robótica con Sincronización en Tiempo Real

Contexto

Un gemelo digital debe reflejar el estado de una celda robótica simulada con baja latencia, y poder reconstruir su historial ante una desconexión.

Descripción del SD

Sistema donde el estado de robots/actuadores se transmite por WebSockets a un servicio de sincronización que mantiene el "gemelo", registra cada cambio como evento (event sourcing) y puede reconstruir el estado en cualquier instante pasado.

Servicios mínimos

1. **Servicio de Celda Física**
 - Emite cambios de estado por WebSocket.
2. **Servicios de Sincronización (Gemelo Digital)**
 - Mantiene el estado vigente en memoria a partir del stream de eventos.
3. **Servicio de Event Store**
 - Persiste cada evento, permitiendo reconstrucción histórica.
4. **Servicio de Consulta**
 - Expone REST para que la UI pida "Estado en el instante T" reconstruido desde el Event Store.

UI obligatoria

- Vista del estado actual del gemelo.
- Control deslizante temporal para ver estados pasados.
- Indicador de conexión en vivo.

Distribución

- Celda física.
- Sincronización + event store.
- Consulta + UI.